

Aula 14 — React Hooks

Módulo: ReactJS + .NET

Carga horária: 4 horas

Projeto base: aula13-react

Objetivo da aula:

Dominar os principais **React Hooks** (`useState` , `useEffect` , `useContext` , `useMemo`) e aplicá-los na construção de um **CRUD completo**, com **formulários controlados**, **organização de estado** e **otimização de performance**.

PARTE 1 — TEORIA (≈ 2 horas)

1 `useState`

◆ O que é?

Hook para **gerenciar estado local** em componentes funcionais.

```
const [contador, setContador] = useState(0);
```

Regras:

- Nunca mutar estado diretamente
 - Atualizações são assíncronas
 - ✓ Base de qualquer CRUD
 - ✓ Atualiza UI automaticamente
-

2 `useEffect`

◆ O que faz?

Executa **efeitos colaterais**:

- Carregamento de dados
- Subscrições
- Timers

```
useEffect(() => { carregarDados(); }, []);
```

Dependências:

- [] → executa 1x
 - [id] → executa quando muda
 - sem array → toda renderização
-

3 useContext

◆ Problema resolvido

Evita **prop drilling** (passar props em cadeia).

◆ Uso

- Criar contexto global
- Compartilhar estado/funções

```
const AppContext = createContext();
```

4 useMemo

◆ Para que serve?

Memorizar cálculos custosos.

```
const total = useMemo(() => calcularTotal(produtos), [produtos]);
```

- ✓ Otimização
 - ✓ Evita reprocessamento desnecessário
-

PARTE 2 — PRÁTICA (≈ 2 horas)

Objetivo Prático

Refatorar o projeto React para:

- CRUD completo
 - Formulários 100% controlados
 - Estado global com Context
 - Otimizações com useMemo
-

1 Criando Contexto Global

 src/context/AppContext.jsx

```
import { createContext, useState } from "react"; export const AppContext = createContext(); export function AppProvider({
children }) { const [produtos, setProdutos] = useState([]); function adicionar(produto) { setProdutos(prev => [...prev,
produto]); } function remover(id) { setProdutos(prev => prev.filter(p => p.id !== id)); } function atualizar(produto) {
setProdutos(prev => prev.map(p => (p.id === produto.id ? produto : p)) ); } return ( <AppContext.Provider value={{ produtos,
adicionar, remover, atualizar }} > {children} </AppContext.Provider> ); }
```

2 Envolvendo a Aplicação com Provider

 main.jsx

```
import React from "react"; import ReactDOM from "react-dom/client"; import App from "./App"; import { AppProvider } from
"./context/AppContext"; ReactDOM.createRoot(document.getElementById("root")).render( <AppProvider> <App /> </AppProvider> );
```

3 CRUD Completo com useContext

 components/Produtos.jsx

```
import { useContext } from "react"; import { AppContext } from "../context/AppContext"; import ProdutoForm from "../ProdutoForm";
import ProdutoList from "../ProdutoList"; function Produtos() { const { produtos } = useContext(AppContext); return ( <div>
<ProdutoForm /> <ProdutoList produtos={produtos} /> </div> ); } export default Produtos;
```

4 Formulário Controlado (Create + Update)

 components/ProdutoForm.jsx

```
import { useState, useContext } from "react"; import { AppContext } from "../context/AppContext"; function ProdutoForm() { const
{ adicionar, atualizar } = useContext(AppContext); const [form, setForm] = useState({ id: null, nome: "", preco: "" }); function
handleChange(e) { setForm({ ...form, [e.target.name]: e.target.value }); } function submit(e) { e.preventDefault(); if
(!form.nome) return; if (form.id) { atualizar(form); } else { adicionar({ ...form, id: Date.now() }); } setForm({ id: null, nome:
"", preco: "" }); } return ( <form onSubmit={submit}> <input name="nome" placeholder="Nome" value={form.nome} onChange=
{handleChange} /> <input name="preco" placeholder="Preço" value={form.preco} onChange={handleChange} /> <button type="submit">
{form.id ? "Atualizar" : "Adicionar"} </button> </form> ); } export default ProdutoForm;
```

- ✓ Controlled Components
- ✓ Um único estado para o formulário

5 Lista com Remoção

 components/ProdutoList.jsx

```
import { useContext } from "react"; import { AppContext } from "../context/AppContext"; function ProdutoList({ produtos }) {  
  const { remover } = useContext(AppContext); return ( <ul> {produtos.map(p => ( <li key={p.id}> {p.nome} - R$ {p.preco} <button  
    onClick={() => remover(p.id)}>Excluir</button> </li> ))} </ul> ); } export default ProdutoList;
```

6 useEffect (Exemplo de Carga Inicial)

```
useEffect(() => { // simula carga inicial adicionar({ id: 1, nome: "Produto Inicial", preco: 100 }); }, []);
```

7 useMemo (Resumo e Otimização)

 components/Resumo.jsx

```
import { useContext, useMemo } from "react"; import { AppContext } from "../context/AppContext"; function Resumo() { const {  
  produtos } = useContext(AppContext); const total = useMemo( () => produtos.reduce((acc, p) => acc + Number(p.preco || 0), 0),  
  [produtos] ); return <h3>Total: R$ {total}</h3>; } export default Resumo;
```

- ✓ Recalcula apenas quando produtos muda
- ✓ Performance otimizada

8 Conceitos Aplicados (Resumo)

Hook	Uso
useState	Formulários
useEffect	Carga inicial
useContext	Estado global
useMemo	Performance

✅ Resultado Esperado

- ✓ CRUD completo com React Hooks
- ✓ Formulários controlados
- ✓ Estado global com Context
- ✓ Código limpo e escalável
- ✓ Base pronta para integração com API .NET

📌 Atividade Avaliativa

👉 Desafio prático:

1. Criar edição por clique
2. Validar preço > 0
3. Criar hook customizado useProdutos
4. Adicionar loading e erro
5. Usar useMemo para filtro